

Prime Factors

CONTENTS

목차

Chapter1

Prime Factors 소개

Chapter2

Prime Factors 프로젝트 준비

Chapter3

Prime Factors TDD 개발

Chapter1

Prime Factors 소개

Prime Factors 소개

- 소인수 분해

양의 정수를 소수들의 곱으로 표현

- 예시 (수를 넣으면, 수인수분해 된 배열이 return 된다)

2->[2]

3->[3]

4->[2,2]

6->[2,3]

8->[2,2,2]

9->[3,3]

12->[2,2,3]

14->[2,7]

- **한번에 전부 구현하지 않는다.**

아주 사소한 것을 조금씩 점진적으로 구현한다.
(하드코딩 해도 된다.)

- **아주 간단한 테스트부터 시작한다.**

그리고 Pass 되도록 구현한다.
이 코드는 아주 특수한 경우에만 동작하게 된다. (하드코딩 했으므로)

- **점차 범용적인 코드로 만든다.**

테스트가 점점 구체화될수록,
코드는 아주 특수한 경우에만 동작하면서
점점 범용적인 코드로 변하게 된다.

테스트 코드도 리팩토링을 해야할까?

- **테스트 코드도 리팩토링을 하는 것이 좋다.**
 - Unit Test 코드는 관리되어야 하기 때문이다.

Chapter2

Prime Factors 프로젝트 준비

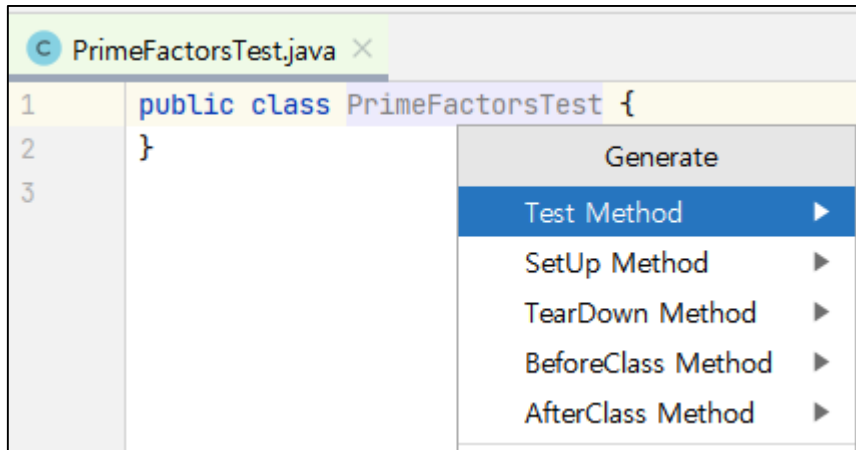
프로젝트, Repository 준비 + 실습을 위한 간단한 Git 복습

@Test 함수 추가

Generate (Alt + Insert) 를 눌러 Test Method 추가하기

해당 단축키를 반드시 암기할 것.
(더 이상 교재에 Generate 단축키 안내 안함)

교안은 JUnit 4로 되어있지만,
JUnit 5로 진행해도 된다.

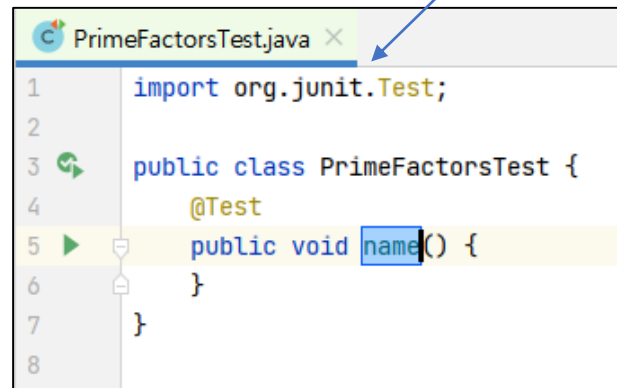


```
1 public class PrimeFactorsTest {
2 }
3
```

Generate

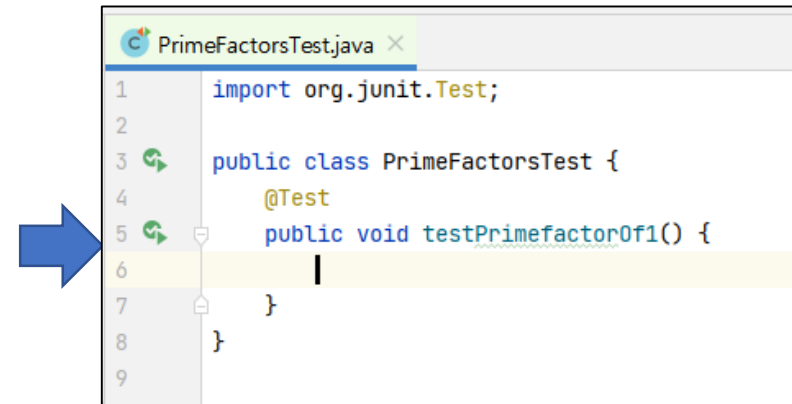
- Test Method
- SetUp Method
- TearDown Method
- BeforeClass Method
- AfterClass Method

Test Method 생성시
import 자동 추가 된다.



```
1 import org.junit.Test;
2
3 public class PrimeFactorsTest {
4     @Test
5     public void name() {
6     }
7 }
8
```

이름 변경하기



```
1 import org.junit.Test;
2
3 public class PrimeFactorsTest {
4     @Test
5     public void testPrimefactorOf1() {
6     }
7 }
8
9
```

변경완료

Class Instance 생성 & 파일 생성

PrimeFactor 의 Instance 생성 후

1. Class 이름에서 Alt + Enter
2. Class 파일 생성

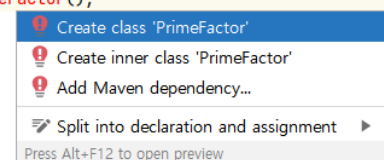
PrimeFactor~~s~~가 아니라, PrimeFactor

```
import org.junit.Test;

public class PrimeFactorsTest {
    @Test
    public void testPrimefactor0f1() {
        PrimeFactor primefactor = new PrimeFactor();
    }
}
```

타이핑 후, 커서를 두고
Alt + Enter 누르기

```
public class PrimeFactorsTest {
    @Test
    public void testPrimefactor0f1() {
        PrimeFactor primefactor = new PrimeFactor();
    }
}
```



The context menu shows options: 'Create class 'PrimeFactor'', 'Create inner class 'PrimeFactor'', 'Add Maven dependency...', and 'Split into declaration and assignment'. The first option is highlighted.

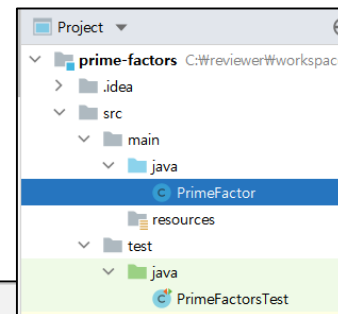
Create Class PrimeFactor

Destination package:

Target destination directory:

OK Cancel

그대로 OK하면
PrimeFactor.java 파일이 자동 생성됨



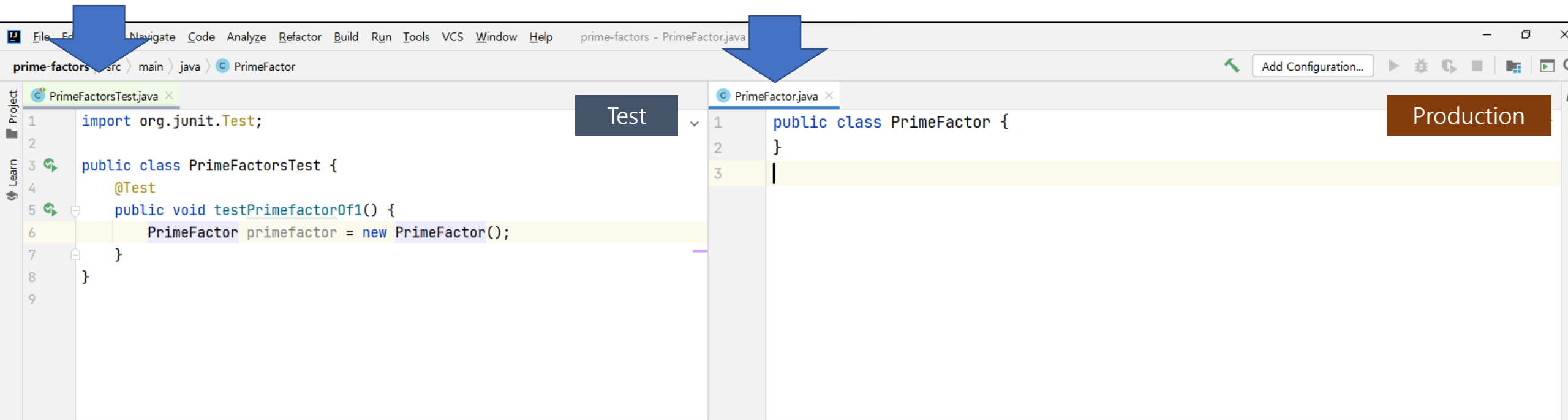
창 배치하기

PrimeFactor.java 파일을 열고 아래와 같이 창 배치

- PrimeFactor.java > 오른쪽버튼 > Split and Move Right

왼쪽 창 : Test 코드 파일

오른쪽 창 : 개발 소스코드 파일



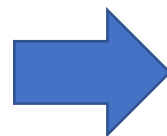
JUnit 정상 테스트 확인

아무런 의미 없는 assertEquals(10, 10) 입력 후
정상적으로 JUnit이 동작하는지 테스트

Test

```
import org.junit.Assert;
import org.junit.Test;

public class PrimeFactorsTest {
    @Test
    public void testPrimefactorOf1() {
        PrimeFactor primefactor = new PrimeFactor();
        Assert.assertEquals(expected: 10, actual: 10);
    }
}
```



```
5
6 public class PrimeFactorsTest {
7     Run 'PrimeFactorsTest' Ctrl+F5
8     Debug 'PrimeFactorsTest'
9     Run 'PrimeFactorsTest' with Coverage
10
11     Modify Run Configuration...
12 }
```

Remote Repository 생성

Github Repository 생성하기

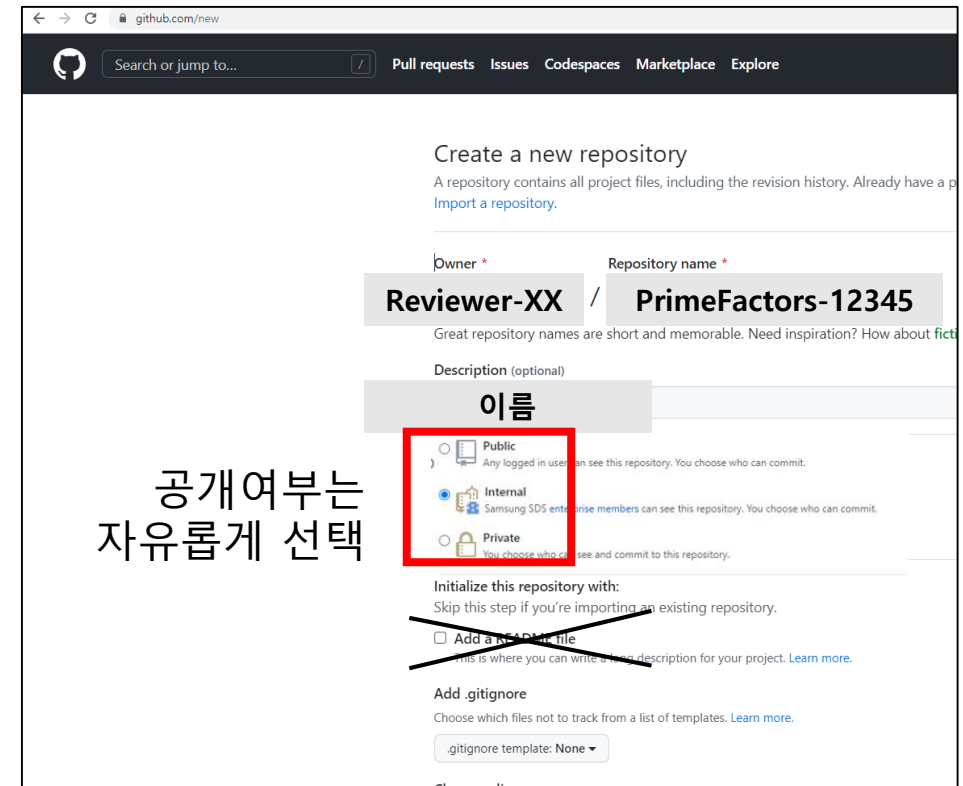
- Repo. name : PrimeFactors-번호
- Description : 이름
- README.md 파일 **생성 안함**

생성 후 설정

- **Setting > Collaborator** : 팀원 추가하기

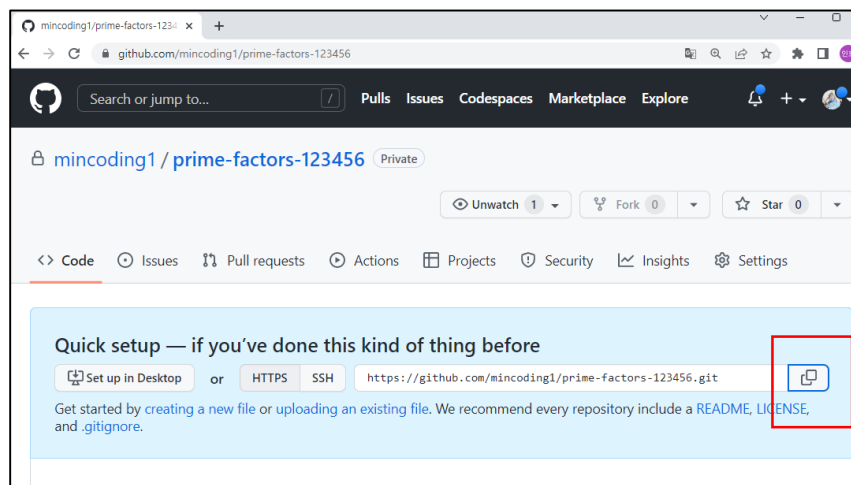
Remote Repository에서는 Collaborator만 코드리뷰어 활동이 가능합니다.

Remote Repository에
팀원들 모두 **Collaborator**로 등록해주세요

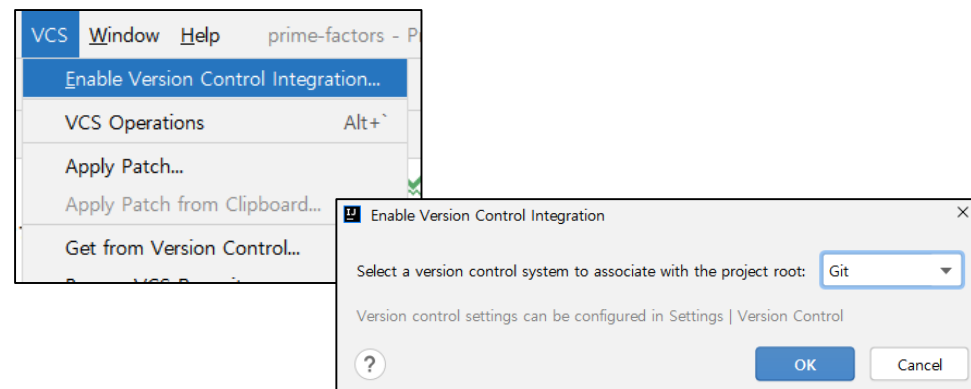


현재까지 내용 Push

- Git GUI 도구를 이용하여 Push 진행



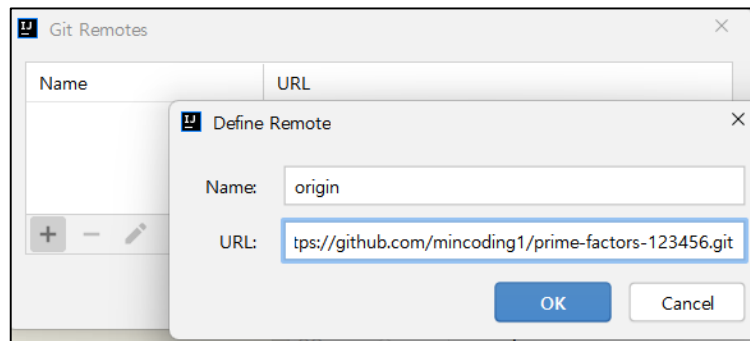
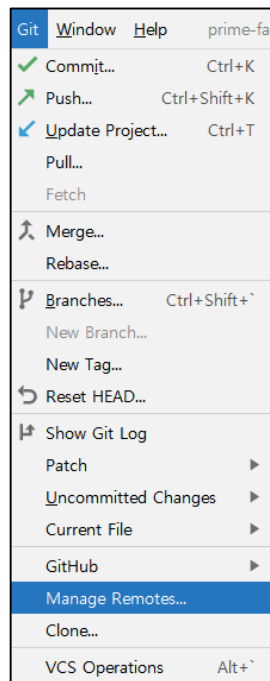
Github에서
Repo 생성 / **Git** 주소 복사하기



IntelliJ 에서,
버전관리시스템으로 Git을 사용하겠다고
지정해주기

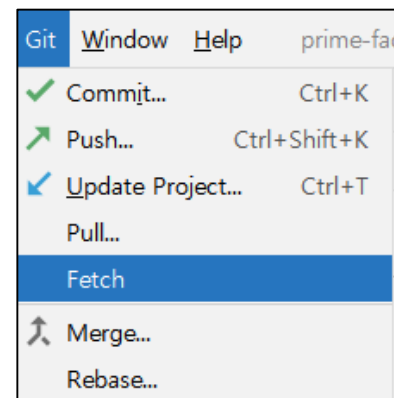
원격 Repository 지정

✓Origin 으로 등록해두기



+ 버튼 클릭 후, 복사한 Git 주소 기입
Remote 이름은 "origin" 으로 하자.

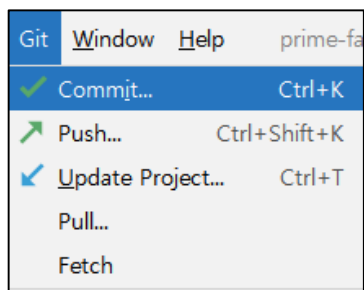
OK 를 눌러 창을 닫음



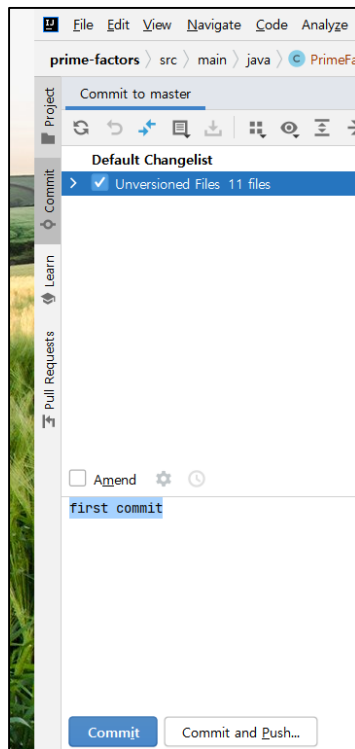
Fetch를 누르면
Remote 정보가
Local Repo로 Sync가 맞춰짐

Commit 하기

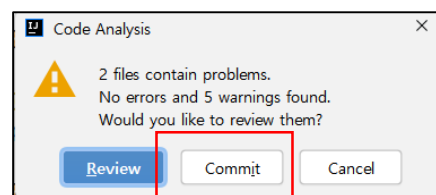
✓Local Repo.로 Commit 하기



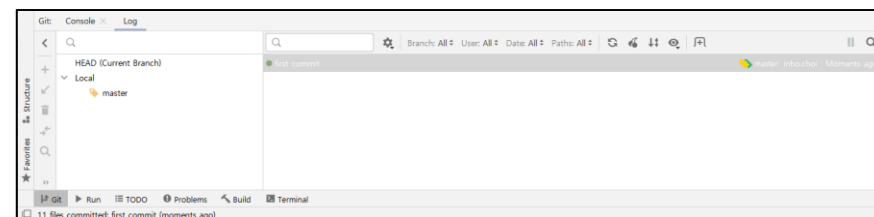
Git > Commit 클릭



변경된 파일 모두 선택하고
"first commit" 이라는 Commit 메시지 입력 후 Commit 클릭



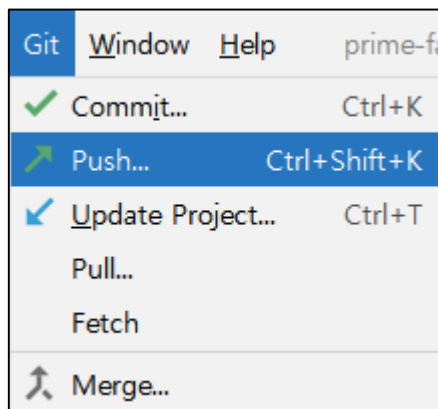
만약 이 창이 뜬다면,
워닝을 무시하고 Commit 하겠다는 의미로
"Commit" 버튼 클릭



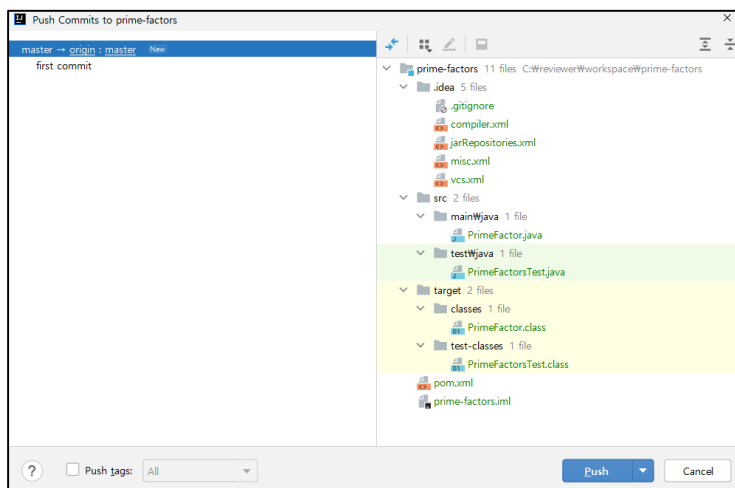
하단, Git 탭을 누르고
[Log] 탭을 한번 더 누르면
Commit 이력을 확인할 수 있음

Push 하기

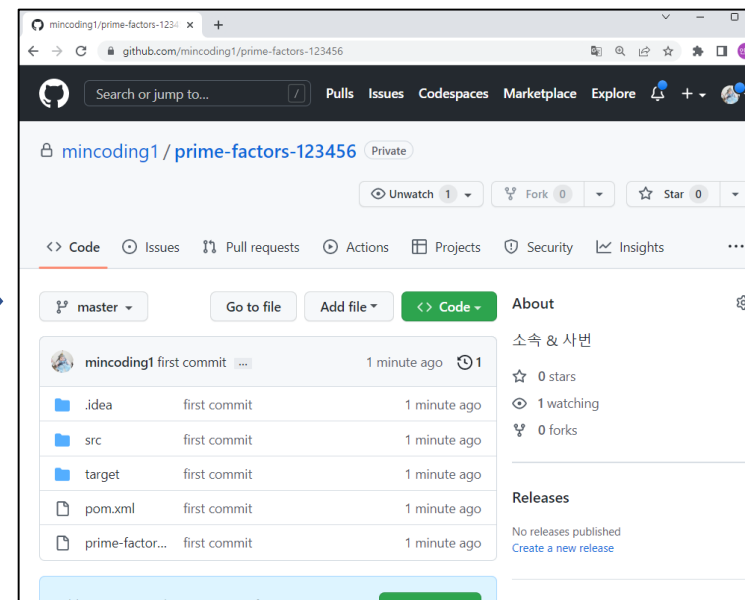
✓Push 하여 결과 확인하기



Git > Push 선택



Push 누르기



Github 에서 결과확인하기

실습 과정 구성

본 실습 과정은 총 7개의 TDD Cycle Step 이 있다.

각 Step 은 다음과 같이 나누어져 있다.

1. Red
2. Green
3. Refactor

프로젝트 제출 방법

- **Commit 타이밍**

1. Red 단계에서는 Commit 하지 않는다.
2. Green 모두 수행 후, Unit Test Pass 시 **Commit** 을 한다.
3. Refactor 모두 수행 후, Unit Test 를 수행하며, Pass 시 **Commit**을 한다.

- **Commit 메시지 헤더 Format**

- Red 단계 : Commit 하지 않는다. (동작되는 코드만 Commit 해야 하기 때문)
- Green 단계 : **[feature] page 번호**
- Refactor 단계 : **[refactoring] page 번호**

- **Push 하여 최종 제출하기**

7 단계 까지 완료 이후, Push 1회만 진행

Chapter3

Prime Factors TDD 개발

Prime Factors 실습(01/07) : TDD Cycle 1

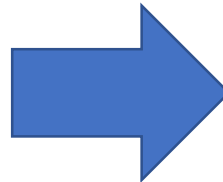
assertEquals문을 완성

그리고 of 함수에 Alt + Enter를 눌러 메서드 자동생성

```
public class PrimeFactorsTest {  
    @Test  
    public void testPrimefactorOf1() {  
        PrimeFactor primefactor = new PrimeFactor();  
        assertEquals(Arrays.asList(), primefactor.of(1));  
    }  
}
```

Test

of(1)의 소인수 분해 결과는
[] 빈 배열이 나오는 것을 기대함



```
ctorOf1() {  
    ctor = new PrimeFactor();  
    .asList(), primefactor.of(1));  
}
```

Create method 'of' in 'PrimeFactor'
Rename reference
Put arguments on separate lines ▶
Press Alt+F12 to open preview

of 라는 메서드를 자동생성하기 위해,
of 에 커서를 두고, Alt + Enter 누르기

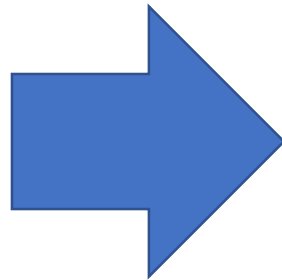
자동생성된 코드 수정

✓Object 형태로 바꿈

빌드는 되지만, Unit Test에서 Fail이 나도록 함

Production

```
PrimeFactorsTest.java x PrimeFactor.java x
1 import java.util.List;
2
3 public class PrimeFactor {
4
5     public <T> List<T> of(int i) {
6     }
7 }
8
```



```
import java.util.List;

public class PrimeFactor {

    public Object of(int i) {
        return null;
    }
}
```

테스트 해보기

- ✓기대값 : Fail 발생 (앞으로는 Run 단축키를 눌러 테스트를 진행할 것)
- ✓Red를 눈으로 확인했으니, Green 단계 진행

The screenshot shows an IDE with two tabs: **Test** (PrimeFactorsTest.java) and **Production** (PrimeFactor.java). The **Test** tab shows the following code:

```
1 import org.junit.Assert;
2 import org.junit.Test;
3
4 import java.util.Arrays;
5
6 public class PrimeFactorsTest {
7     @Test
8     public void testPrimefactorOf1() {
9         PrimeFactor primefactor = new PrimeFactor();
10        Assert.assertEquals(Arrays.asList(), primefactor.of(1));
11    }
12 }
13
14 }
```

The **Production** tab shows the following code:

```
1 import java.util.List;
2
3 public class PrimeFactor {
4     public Object of(int i) {
5         return null;
6     }
7 }
8
```

The **Run** output shows the following error:

```
Run: PrimeFactorsTest
Tests failed: 1 of 1 test - 9 ms
PrimeFactorsTest 9 ms
  testPrimefactorOf1 9 ms
    java.lang.AssertionError:
      Expected :[]
      Actual   :null
      <Click to see difference>
```

빈 배열을 기대했지만,
null 값이 리턴되어 Fail 발생

Production 코드 수정

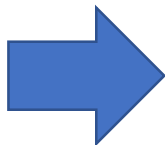
✓Unit Test에 통과될 수준만큼만 작성

```
import java.util.List;

public class PrimeFactor {

    public Object of(int i) {
        return null;
    }
}
```

Production



```
import java.util.ArrayList;
import java.util.List;

public class PrimeFactor {
    public List<Integer> of(int i) {
        List<Integer> factors = new ArrayList<>();
        return factors;
    }
}
```

Production

어떤 수를 넣던,
무조건 [] 빈 배열이 나오는 것으로 변경

결과확인

테스트를 해보고 Pass를 확인

The screenshot displays an IDE with two Java files and a Run console window.

PrimeFactorsTest.java

```
1 import org.junit.Assert;
2 import org.junit.Test;
3
4 import java.util.Arrays;
5
6 public class PrimeFactorsTest {
7     @Test
8     public void testPrimefactorOf1() {
9         PrimeFactor primefactor = new PrimeFactor();
10        Assert.assertEquals(Arrays.asList(), primefactor.of(1));
11    }
12 }
13
14
```

PrimeFactor.java

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class PrimeFactor {
5     public List<Integer> of(int i) {
6         List<Integer> factors = new ArrayList<>();
7         return factors;
8     }
9 }
10
```

Run Console

Run: PrimeFactorsTest

Tests passed: 1 of 1 test – 6 ms

Test Name	Duration
PrimeFactorsTest	6 ms
testPrimefactorOf1	6 ms

Process finished with exit code 0

Refactor 단계

따로 Clean 코드로 만들 내용이 없으므로, 생략함.

이렇게 TDD한 Cycle을 완료함

(코드 수정한 내용이 없으므로, Commit 안함)

```
import java.util.ArrayList;
import java.util.List;

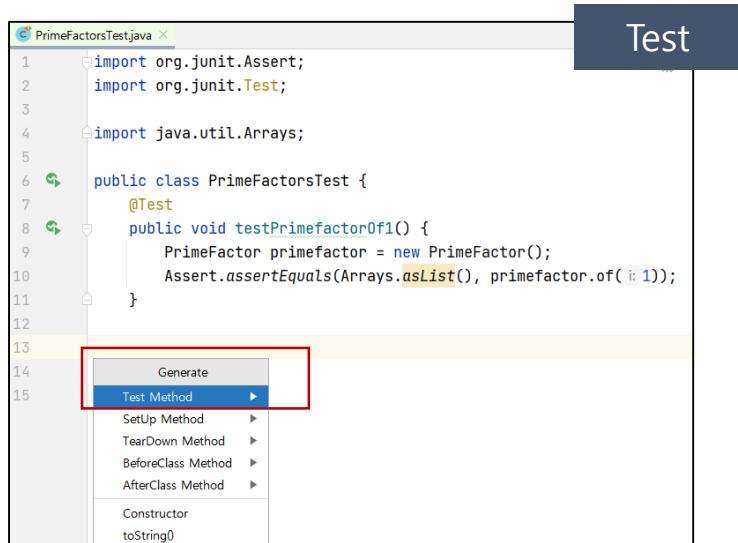
public class PrimeFactor {
    public List<Integer> of(int i) {
        List<Integer> factors = new ArrayList<>();
        return factors;
    }
}
```

Production

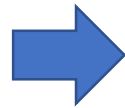
Clean 코드로 변경할 내용이 없음

Prime Factors 실습(02/07) : TDD Cycle 2

✓Red – '2'를 소인수분해 하는 테스트케이스 작성



```
1 import org.junit.Assert;
2 import org.junit.Test;
3
4 import java.util.Arrays;
5
6 public class PrimeFactorsTest {
7     @Test
8     public void testPrimefactorOf1() {
9         PrimeFactor primefactor = new PrimeFactor();
10        Assert.assertEquals(Arrays.asList(), primefactor.of(1));
11    }
12
13    // Generate
14    // Test Method
15    // SetUp Method
16    // TearDown Method
17    // BeforeClass Method
18    // AfterClass Method
19    // Constructor
20    // toString()
```



```
public class PrimeFactorsTest {
    @Test
    public void testPrimefactorOf1() {
        PrimeFactor primefactor = new PrimeFactor();
        Assert.assertEquals(Arrays.asList(), primefactor.of(1));
    }

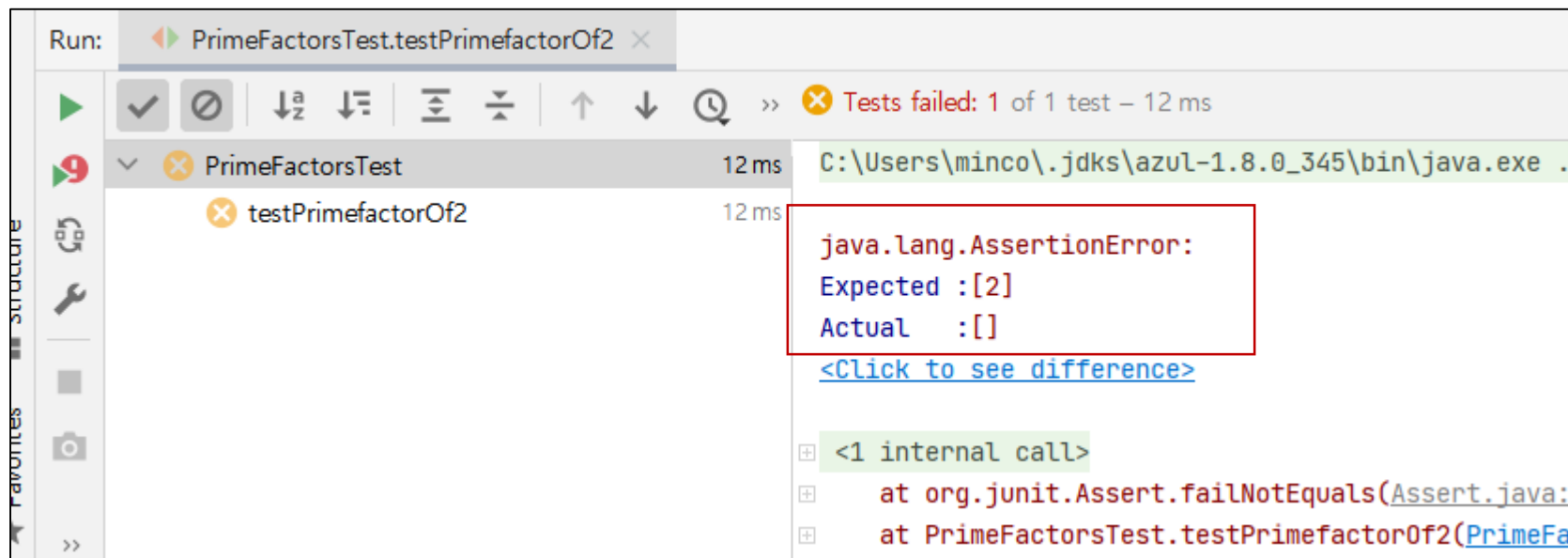
    @Test
    public void testPrimefactorOf2() {
        PrimeFactor primefactor = new PrimeFactor();
        Assert.assertEquals(Arrays.asList(2), primefactor.of(2));
    }
}
```

testPrimefactorOf1 코드를 복사 붙여넣기 하여,
수정 진행
코드 입력 후 Red를 눈으로 확인

Fail 확인하기

✓기대값과 달라 Fail을 눈으로 확인하기

- 이제 Pass 될 정도로만 구현을 시작하자.



결과가 [2] 가 나와야 하는데,
실제 값은 빈 배열이 나왔으므로 Fail

Green 단계 시작

✓ Pass가 될 정도로만 구현하기

- 이름 바꾸기 단축키 : Shift + F6
- 꼭 암기할 것. (더 이상 교재에 이름 바꾸기 단축키 안내 안함)

Production

```
public class PrimeFactor {  
    public List<Integer> of(int i) {  
        List<Integer> factors = new ArrayList<>();  
        if (i == 2) {  
            factors.add(2);  
        }  
        return factors;  
    }  
}
```

위 코드 넣기

```
public class PrimeFactor {  
    public List<Integer> of(int i) {  
        List<Integer> factors = 1  
        if (i == 2) {  
            factors.add(2);  
        }  
        return factors;  
    }  
}
```

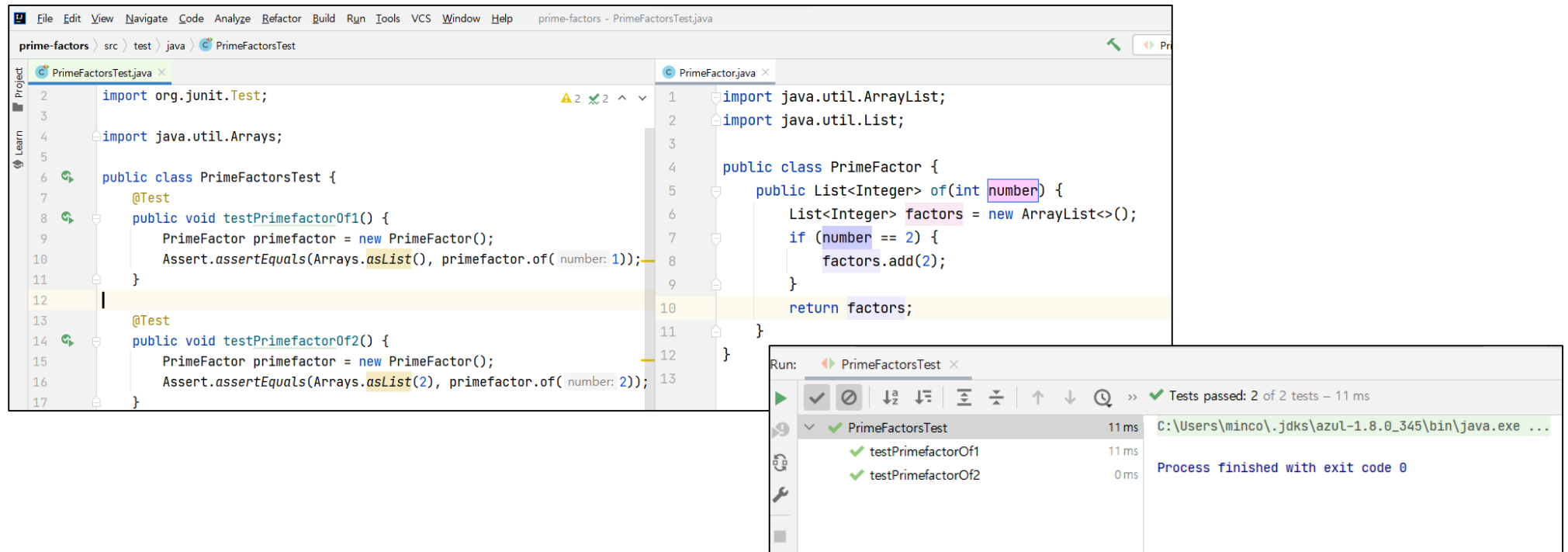
변수명 바꾸기 (이름바꾸기)
단축키 : Shift + F6 누르고
원하는 이름 기입하기

```
import java.util.ArrayList;  
import java.util.List;  
  
public class PrimeFactor {  
    public List<Integer> of(int number) {  
        List<Integer> factors = new ArrayList<>();  
        if (number == 2) {  
            factors.add(2);  
        }  
        return factors;  
    }  
}
```

변수명을 number로 변경

테스트 수행하기

✓Green을 보았으니, 이제 Refactor 단계 진행



The screenshot displays an IDE with two open Java files: `PrimeFactorsTest.java` and `PrimeFactor.java`. The `PrimeFactorsTest.java` file contains two JUnit tests: `testPrimefactorOf1()` and `testPrimefactorOf2()`. The `PrimeFactor.java` file contains a `PrimeFactor` class with an `of(int number)` method. The `Run` window at the bottom right shows the execution results for `PrimeFactorsTest`, indicating that both tests passed successfully.

```
prime-factors - PrimeFactorsTest.java

File Edit View Navigate Code Analyze Refactor Build Run Tools VCS Window Help

prime-factors > src > test > java > PrimeFactorsTest

PrimeFactorsTest.java x
2 import org.junit.Test;
3
4 import java.util.Arrays;
5
6 public class PrimeFactorsTest {
7     @Test
8     public void testPrimefactorOf1() {
9         PrimeFactor primefactor = new PrimeFactor();
10        Assert.assertEquals(Arrays.asList(), primefactor.of( number: 1));
11    }
12
13    @Test
14    public void testPrimefactorOf2() {
15        PrimeFactor primefactor = new PrimeFactor();
16        Assert.assertEquals(Arrays.asList(2), primefactor.of( number: 2));
17    }
18 }

PrimeFactor.java x
1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class PrimeFactor {
5     public List<Integer> of(int number) {
6         List<Integer> factors = new ArrayList<>();
7         if (number == 2) {
8             factors.add(2);
9         }
10        return factors;
11    }
12 }
13 }
```

Run: PrimeFactorsTest x

Test Name	Duration	Exit Code
PrimeFactorsTest	11 ms	0
testPrimefactorOf1	11 ms	0
testPrimefactorOf2	0 ms	0

Tests passed: 2 of 2 tests - 11 ms

Process finished with exit code 0

리팩토링 단계 - 중복 제거하기

- ✓ Production Code는 리팩토링 할 내용이 없음
 - Test Code에서 중복제거 진행

Test

```
public class PrimeFactorsTest {  
    @Test  
    public void testPrimefactor0f1() {  
        PrimeFactor primefactor = new PrimeFactor();  
        Assert.assertEquals(Arrays.asList(), primefactor.of( number: 1));  
    }  
  
    @Test  
    public void testPrimefactor0f2() {  
        PrimeFactor primefactor = new PrimeFactor();  
        Assert.assertEquals(Arrays.asList(2), primefactor.of( number: 2));  
    }  
}
```

매번 테스트 할 때마다
생성해주는 코드 중복 제거



```
import java.util.Arrays;  
  
public class PrimeFactorsTest {  
    @Test  
    public void testPrimefactor0f1() {  
        PrimeFactor primefactor = new PrimeFactor();  
        Assert.assertEquals(Arrays.asList(), primefactor.of( number: 1));  
    }  
  
    @Test  
    public void testPrimefactor0f2() {  
        PrimeFactor primefactor = new PrimeFactor();  
        Assert.assertEquals(Arrays.asList(2), primefactor.of( number: 2));  
    }  
}
```

Generate
Test Method
Setup Method
TearDown Method
BeforeClass Method
AfterClass Method
Constructor
toString()
Override Methods... Ctrl+O
Copyright

매번 테스트할 때 마다
동작하는 메서드 (SetUp) 생성하기



```
public class PrimeFactorsTest {  
    @Before  
    public void setUp() throws Exception {  
    }  
  
    @Test  
    public void testPrimefactor0f1() {  
        PrimeFactor primefactor = new PrimeFactor();  
        Assert.assertEquals(Arrays.asList(), primefactor.of( number: 1));  
    }  
  
    @Test  
    public void testPrimefactor0f2() {  
        PrimeFactor primefactor = new PrimeFactor();  
        Assert.assertEquals(Arrays.asList(2), primefactor.of( number: 2));  
    }  
}
```

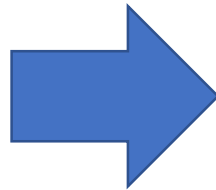
@Before 어노테이션과
함께 메서드 생성 완료

중복 제거하기

✓Refactoring – 중복제거 (Convert local variable to field)

Test

```
public class PrimeFactorsTest {  
    @Before  
    public void setUp() throws Exception {  
    }  
  
    @Test  
    public void testPrimefactorOf1() {  
        PrimeFactor primefactor = new PrimeFactor();  
        Assert.assertEquals(Arrays.asList(), primefactor.of(1));  
    }  
  
    @Test  
    public void testPrimefactorOf2() {  
        PrimeFactor primefactor = new PrimeFactor();  
        Assert.assertEquals(Arrays.asList(2), primefactor.of(2));  
    }  
}
```



```
public class PrimeFactorsTest {  
    PrimeFactor primefactor;  
  
    @Before  
    public void setUp() throws Exception {  
        primefactor = new PrimeFactor();  
    }  
  
    @Test  
    public void testPrimefactorOf1() {  
        Assert.assertEquals(Arrays.asList(), primefactor.of(1));  
    }  
  
    @Test  
    public void testPrimefactorOf2() {  
        Assert.assertEquals(Arrays.asList(2), primefactor.of(2));  
    }  
}
```

리팩토링이 잘 되었는지 Unit Test 하기

✓ Pass 되었다면, 한 Cycle이 완료 된 것.

The screenshot displays an IDE with two Java files open: `PrimeFactorsTest.java` and `PrimeFactor.java`. The `PrimeFactorsTest` class includes a `@Before` method to initialize a `PrimeFactor` instance and two `@Test` methods, `testPrimefactorOf1()` and `testPrimefactorOf2()`, both using `Assert.assertEquals` to verify the output of `primefactor.of()`. The `PrimeFactor` class has a `of(int number)` method that returns a `List<Integer>` of prime factors. The `Run` window at the bottom right shows that both tests passed successfully, with a total execution time of 6 ms. The command used to run the tests is `C:\Users\minco\.jdk\azul-1.8.0_345\bin\java.exe ...`.

```
prime-factors - PrimeFactorsTest.java
prime-factors \src \test \java \PrimeFactorsTest

import java.util.Arrays;

public class PrimeFactorsTest {

    PrimeFactor primefactor;

    @Before
    public void setUp() throws Exception {
        primefactor = new PrimeFactor();
    }

    @Test
    public void testPrimefactorOf1() {
        Assert.assertEquals(Arrays.asList(), primefactor.of(1));
    }

    @Test
    public void testPrimefactorOf2() {
        Assert.assertEquals(Arrays.asList(2), primefactor.of(2));
    }
}

PrimeFactor.java

import java.util.ArrayList;
import java.util.List;

public class PrimeFactor {

    public List<Integer> of(int number) {
        List<Integer> factors = new ArrayList<>();
        if (number == 2) {
            factors.add(2);
        }
        return factors;
    }
}
```

Run: PrimeFactorsTest

Tests passed: 2 of 2 tests - 6 ms

Test Name	Duration
PrimeFactorsTest	6 ms
testPrimefactorOf1	6 ms
testPrimefactorOf2	0 ms

Process finished with exit code 0

Prime Factors 실습(03/07) : TDD Cycle 3

✓Red – '3'을 소인수분해 하는 테스트케이스 작성

```
PrimeFactor primefactor;

@Before
public void setUp() throws Exception {
    primefactor = new PrimeFactor();
}

@Test
public void testPrimefactorOf1() {
    Assert.assertEquals(Arrays.asList(), primefactor.of( number: 1));
}

@Test
public void testPrimefactorOf2() {
    Assert.assertEquals(Arrays.asList(2), primefactor.of( number: 2));
}

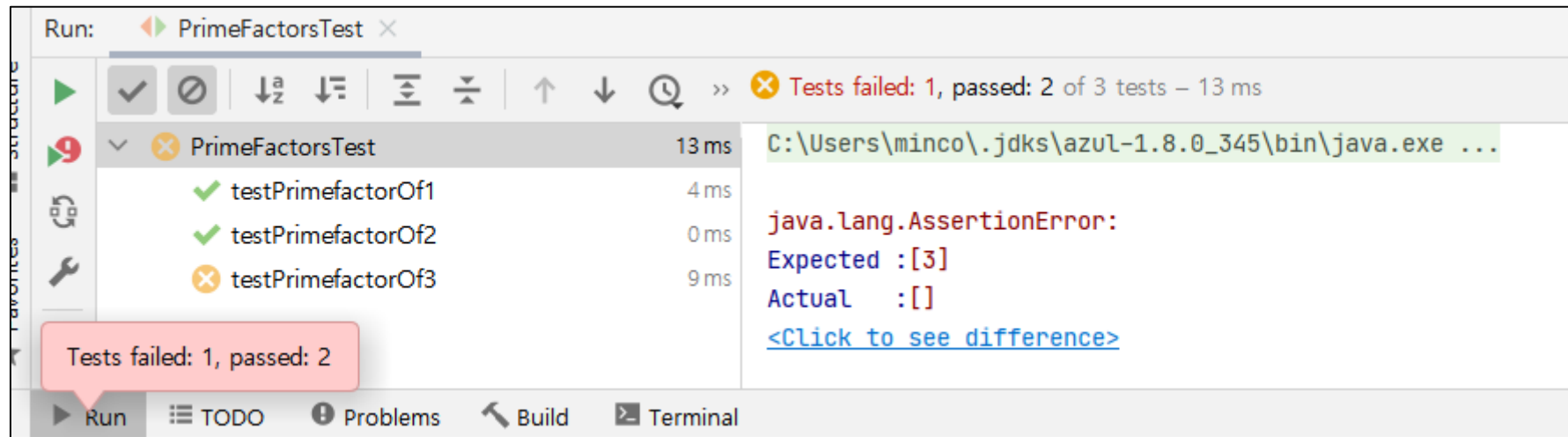
@Test
public void testPrimefactorOf3() {
    Assert.assertEquals(Arrays.asList(3), primefactor.of( number: 3));
}
```

Alt + Insert 로 Test 코드 제너레이트를 하거나,
위 코드를 복사 붙여넣기 진행.

테스트 내용 : 소인수 3을 넣었을 때 [3] 이 나와야 함.

Fail 확인하기

✓Fail을 눈으로 확인하였으니, 이제 Green 단계를 진행함



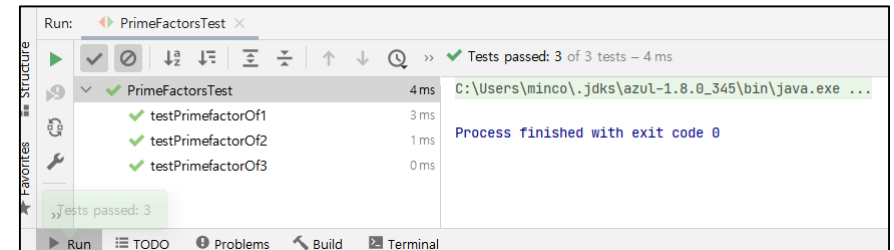
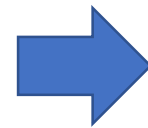
Fail을 눈으로 확인함

Test에 성공할 정도로만 구현하기

✓Green – 테스트케이스 성공

```
public class PrimeFactor {  
    public List<Integer> of(int number) {  
        List<Integer> factors = new ArrayList<>();  
        if (number == 2) {  
            factors.add(2);  
        }  
        else if (number == 3) {  
            factors.add(3);  
        }  
        return factors;  
    }  
}
```

Production



코드를 추가하고,
Green을 눈으로 확인

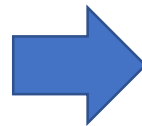
리팩토링 - 테스트 코드

✓Production Code를 Refactoring 함

Production

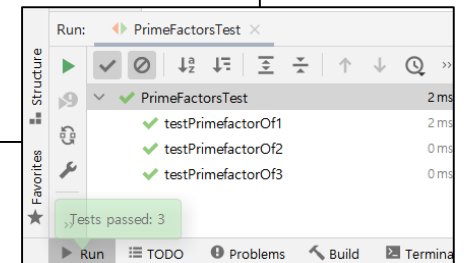
```
public class PrimeFactor {  
    public List<Integer> of(int number) {  
        List<Integer> factors = new ArrayList<>();  
        if (number == 2) {  
            factors.add(2);  
        }  
        else if (number == 3) {  
            factors.add(3);  
        }  
        return factors;  
    }  
}
```

하드코딩 일부를 number 변수로 변경



```
public class PrimeFactor {  
    public List<Integer> of(int number) {  
        List<Integer> factors = new ArrayList<>();  
        if (number == 2) {  
            factors.add(number);  
        }  
        else if (number == 3) {  
            factors.add(number);  
        }  
        return factors;  
    }  
}
```

리팩토링 후 반드시 Test로 결과 확인



리팩토링 - 더 Generalize 하도록 개선

✓Refactoring - generalize

- 조금 더 범용적인 코드로 리팩토링함
- 여기까지 TDD한 Cycle 완료

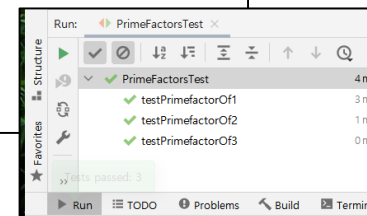
Production

```
public class PrimeFactor {  
    public List<Integer> of(int number) {  
        List<Integer> factors = new ArrayList<>();  
        if (number == 2) {  
            factors.add(number);  
        }  
        else if (number == 3) {  
            factors.add(number);  
        }  
        return factors;  
    }  
}
```



```
public class PrimeFactor {  
    public List<Integer> of(int number) {  
        List<Integer> factors = new ArrayList<>();  
        if (number > 1) {  
            factors.add(number);  
        }  
        return factors;  
    }  
}
```

리팩토링 후 반드시 Test로 결과 확인



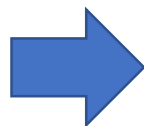
Prime Factors 실습(04/07) : TDD Cycle 4

✓ Red – '4'를 소인수분해 하는 테스트케이스 작성

Test

```
public void testPrimefactorOf2() {  
    Assert.assertEquals(Arrays.asList(2), primefactor.of( number: 2));  
}  
  
@Test  
public void testPrimefactorOf3() {  
    Assert.assertEquals(Arrays.asList(3), primefactor.of( number: 3));  
}  
  
@Test  
public void testPrimefactorOf4() {  
    Assert.assertEquals(Arrays.asList(2,2), primefactor.of( number: 4));  
}  
}
```

4를 소인수 분해하면, [2, 2] 가 나와야하는
테스트 코드 추가하기



Run: PrimeFactorsTest

Tests failed: 1, passed: 3 of 4 tests

PrimeFactorsTest	13 ms	
testPrimefactorOf1	4 ms	✓
testPrimefactorOf2	0 ms	✓
testPrimefactorOf3	1 ms	✓
testPrimefactorOf4	8 ms	✗

java.lang.AssertionError:
Expected :[2, 2]
Actual :[4]
[Click to see difference](#)

Fail을 눈으로 확인

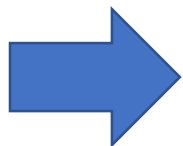
4일 때, 소인수 분해가 되도록 코드 추가

✓Green – 테스트케이스 성공

Production

```
public class PrimeFactor {  
    public List<Integer> of(int number) {  
        List<Integer> factors = new ArrayList<>();  
        if (number > 1) {  
            factors.add(number);  
        }  
        return factors;  
    }  
}
```

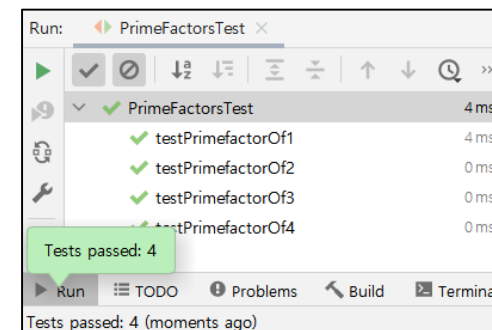
기존 소스코드



```
import java.util.ArrayList;  
import java.util.List;  
  
public class PrimeFactor {  
    public List<Integer> of(int number) {  
        List<Integer> factors = new ArrayList<>();  
        if (number > 1) {  
            if (number == 4) {  
                factors.add(2);  
                factors.add(2);  
            }  
            else {  
                factors.add(number);  
            }  
        }  
        return factors;  
    }  
}
```

코드 추가하기

4인 경우, [2, 2]가 되고,
그 외는 기존처럼 [number]가 되도록 함



Green을 눈으로 확인

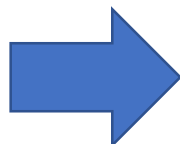
리팩토링 - 더 Generalize하도록 코드 수정

✓Refactoring - generalize

Production

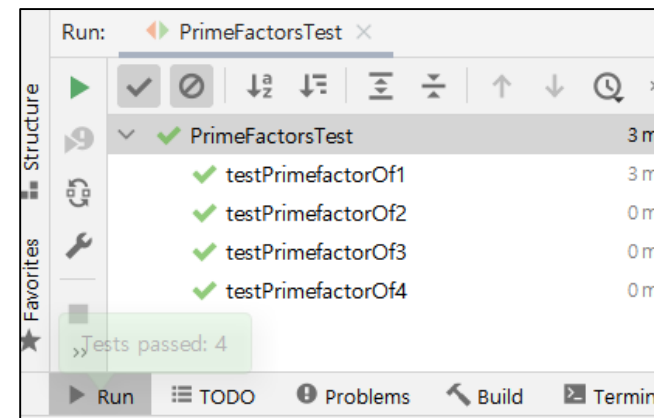
```
public class PrimeFactor {  
    public List<Integer> of(int number) {  
        List<Integer> factors = new ArrayList<>();  
        if (number > 1) {  
            if (number == 4) {  
                factors.add(2);  
                factors.add(2);  
            }  
            else {  
                factors.add(number);  
            }  
        }  
        return factors;  
    }  
}
```

기존 소스코드



```
public class PrimeFactor {  
    public List<Integer> of(int number) {  
        List<Integer> factors = new ArrayList<>();  
        if (number > 1) {  
            if (number == 4) {  
                if (number % 2 == 0) {  
                    factors.add(2);  
                    number /= 2;  
                }  
                if (number % 2 == 0) {  
                    factors.add(2);  
                    number /= 2;  
                }  
            }  
            else {  
                factors.add(number);  
            }  
        }  
        return factors;  
    }  
}
```

조금 더 범용적이도록
소스코드 리팩토링



리팩토링 후에도,
이상없음을 눈으로 확인

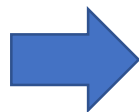
리팩토링 - if를 while로 변경

- ✓ Refactoring - if 반복구문 리팩토링
 - 여기까지 하나의 TDD Cycle을 마무리

Production

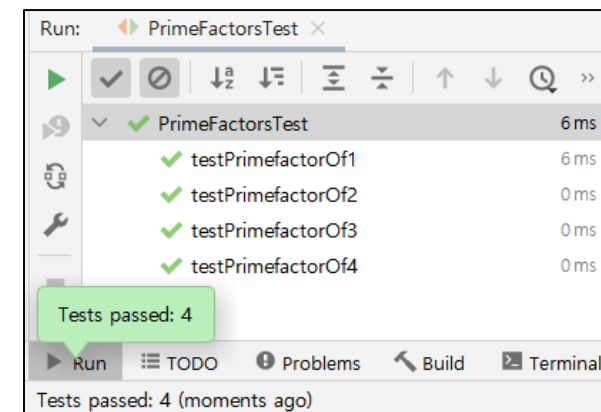
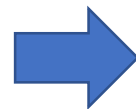
```
public class PrimeFactor {  
    public List<Integer> of(int number) {  
        List<Integer> factors = new ArrayList<>();  
        if (number > 1) {  
            if (number == 4) {  
                if (number % 2 == 0) {  
                    factors.add(2);  
                    number /= 2;  
                }  
                if (number % 2 == 0) {  
                    factors.add(2);  
                    number /= 2;  
                }  
            }  
            else {  
                factors.add(number);  
            }  
        }  
        return factors;  
    }  
}
```

기존 소스코드



```
public class PrimeFactor {  
    public List<Integer> of(int number) {  
        List<Integer> factors = new ArrayList<>();  
        if (number > 1) {  
            if (number == 4) {  
                while (number % 2 == 0) {  
                    factors.add(2);  
                    number /= 2;  
                }  
            }  
            else {  
                factors.add(number);  
            }  
        }  
        return factors;  
    }  
}
```

조금 더 범용적이도록
소스코드 리팩토링



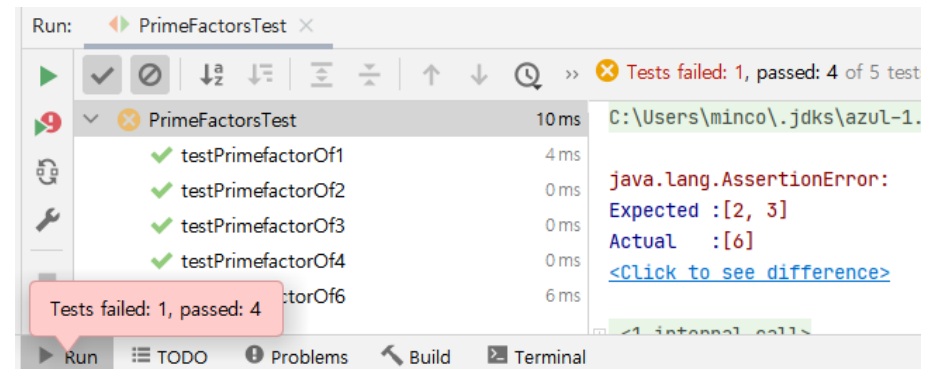
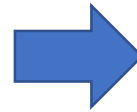
리팩토링 후에도,
이상없음을 눈으로 확인

Prime Factors 실습(05/07) : TDD Cycle 5

✓ Red – '6'을 소인수분해 하는 테스트케이스 작성

```
public void testPrimefactorOf3() {  
    Assert.assertEquals(Arrays.asList(3), primefactor.of( number: 3));  
}  
  
@Test  
public void testPrimefactorOf4() {  
    Assert.assertEquals(Arrays.asList(2,2), primefactor.of( number: 4));  
}  
  
@Test  
public void testPrimefactorOf6() {  
    Assert.assertEquals(Arrays.asList(2,3), primefactor.of( number: 6));  
}
```

Test



6에 대한 소인수 분해 값인 [2, 3] 기대값 테스트

Fail을 눈으로 확인

Green – 6의 소인수 분해 정답 나오도록 코드 추가

✓Green – 테스트케이스 성공

```
public class PrimeFactor {  
    public List<Integer> of(int number) {  
        List<Integer> factors = new ArrayList<>();  
        if (number > 1) {  
            if (number == 4) {  
                while (number % 2 == 0) {  
                    factors.add(2);  
                    number /= 2;  
                }  
            }  
            else {  
                factors.add(number);  
            }  
        }  
        return factors;  
    }  
}
```

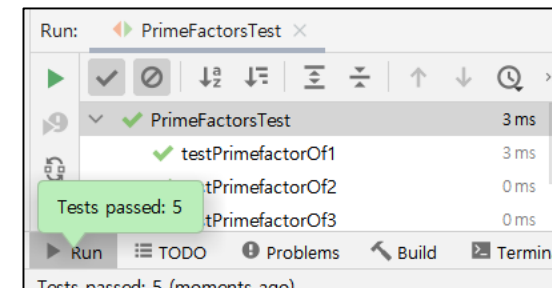
기존 소스코드

Production



```
public class PrimeFactor {  
    public List<Integer> of(int number) {  
        List<Integer> factors = new ArrayList<>();  
        if (number > 1) {  
            if (number == 4) {  
                while (number % 2 == 0) {  
                    factors.add(2);  
                    number /= 2;  
                }  
            }  
            else if (number == 6) {  
                factors.add(2);  
                factors.add(3);  
            }  
            else {  
                factors.add(number);  
            }  
        }  
        return factors;  
    }  
}
```

6일때 [2, 3]이 나오도록 코드 추가



정상 동작함을 눈으로 확인

리팩토링 - 조금 더 범용적이도록

✓Refactoring – generalize

Production

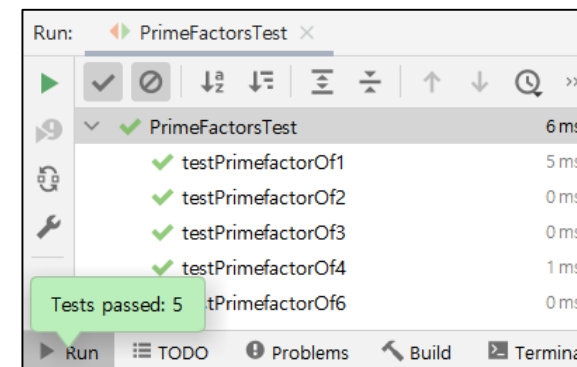
```
public class PrimeFactor {
    public List<Integer> of(int number) {
        List<Integer> factors = new ArrayList<>();
        if (number > 1) {
            if (number == 4) {
                while (number % 2 == 0) {
                    factors.add(2);
                    number /= 2;
                }
            }
            else if (number == 6) {
                factors.add(2);
                factors.add(3);
            }
            else {
                factors.add(number);
            }
        }
        return factors;
    }
}
```

기존 소스코드



```
public class PrimeFactor {
    public List<Integer> of(int number) {
        List<Integer> factors = new ArrayList<>();
        if (number > 1) {
            if (number == 4) {
                while (number % 2 == 0) {
                    factors.add(2);
                    number /= 2;
                }
            }
            else if (number == 6) {
                while (number % 2 == 0) {
                    factors.add(2);
                    number /= 2;
                }
                while (number % 3 == 0) {
                    factors.add(3);
                    number /= 3;
                }
            }
            else {
                factors.add(number);
            }
        }
        return factors;
    }
}
```

조금 더 범용적이도록
소스코드 리팩토링



리팩토링 후에도,
이상 없음을 눈으로 확인

리팩토링 – Extract local variable (변수 추출하기)

✓Refactoring – extract local variable

- Refactor 메뉴 단축키 : **Ctrl + Alt + Shift + T** (암기 할 것!)

Production

```

1  import java.util.List;
2
3
4  public class PrimeFactor {
5      public List<Integer> of(int number) {
6          List<Integer> factors = new ArrayList<>();
7          if (number > 1) {
8              if (number == 4) {
9                  while (number % 2 == 0) {
10                     factors.add(2);
11                     number /= 2;
12                 }
13             }
14             else if (number == 6) {
15                 while (number % 2 == 0) {
16                     factors.add(2);
17                     number /= 2;
18                 }
19                 while (number % 3 == 0) {
20                     factors.add(3);
21                     number /= 3;
22                 }
23             }
24             else {

```

Refactor This

Extract/Introduce

3. Introduce Variable... Ctrl+Alt+V

Multiple occurrences found

Replace this occurrence only

Replace 3 occurrences in 'if-then' block

Replace all 6 occurrences

➡

```

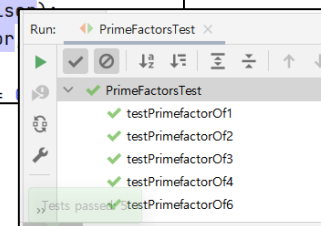
3
4  public class PrimeFactor {
5      public List<Integer> of(int number) {
6          List<Integer> factors = new ArrayList<>();
7          if (number > 1) {
8              int divisor = 2;
9              if (number % divisor == 0) {
10                 factors.add(divisor);
11                 number /= divisor;
12             }
13             else if (number == 6) {
14                 while (number % divisor == 0) {
15                     factors.add(divisor);
16                     number /= divisor;
17                 }
18                 while (number % 3 == 0) {
19                     factors.add(3);
20                     number /= 3;
21                 }
22             }
23             else {

```

숫자 2 대신, 변수 값으로 이름을 붙여줌

변수 추출하기 / 변수 도입하기
(extract local variable / Introduce Variable)

변수 이름 : divisor 후
리팩토링 잘 되었는지
테스트 진행



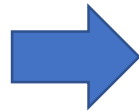
리팩토링 - divisor로 변경

✓숫자 3 대신 divisor 값으로 변경

Production

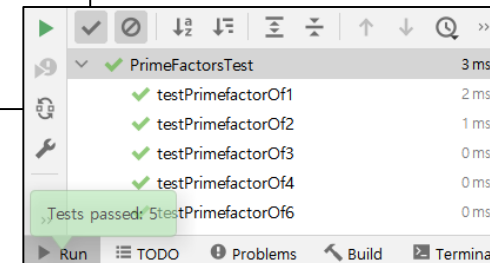
```
else if (number == 6) {  
    while (number % divisor == 0) {  
        factors.add(divisor);  
        number /= divisor;  
    }  
    while (number % 3 == 0) {  
        factors.add(3);  
        number /= 3;  
    }  
}  
else {  
    factors.add(number);  
}
```

기존 소스코드



```
else if (number == 6) {  
    while (number % divisor == 0) {  
        factors.add(divisor);  
        number /= divisor;  
    }  
    divisor++;  
    while (number % divisor == 0) {  
        factors.add(divisor);  
        number /= divisor;  
    }  
    divisor++;  
}  
else {  
    factors.add(number);  
}
```

divisor + 1 로 해당 코드 수정
그리고 테스트



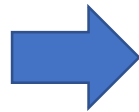
리팩토링 - for loop로 변경

- ✓ Refactoring - 비슷한 중복을 완전한 중복으로 변환
 - 이렇게 하나의 TDD Cycle 완료

Production

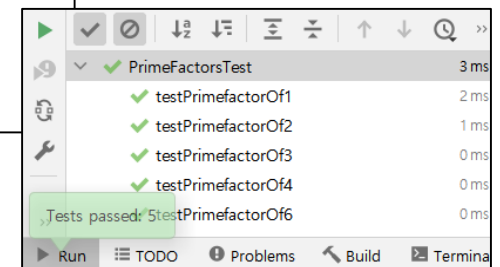
```
else if (number == 6) {  
    while (number % divisor == 0) {  
        factors.add(divisor);  
        number /= divisor;  
    }  
    divisor++;  
    while (number % divisor == 0) {  
        factors.add(divisor);  
        number /= divisor;  
    }  
    divisor++;  
}  
else {  
    factors.add(number);  
}
```

기존 소스코드



```
else if (number == 6) {  
    for (divisor = 2; number > 1; divisor++) {  
        while (number % divisor == 0) {  
            factors.add(divisor);  
            number /= divisor;  
        }  
    }  
}  
else {  
    factors.add(number);  
}
```

for문 divisor / number 순서 유의하여
코드 작성하기



Prime Factors 실습(06/07) : TDD Cycle 6

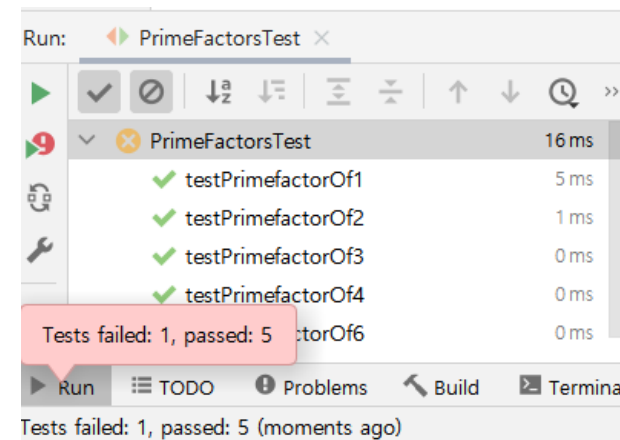
✓ Red – '9'를 소인수분해 하는 테스트케이스 작성

Test

```
@Test
public void testPrimefactorOf6() {
    Assert.assertEquals(Arrays.asList(2,3), primefactor.of( number: 6));
}

@Test
public void testPrimefactorOf9() {
    Assert.assertEquals(Arrays.asList(3,3), primefactor.of( number: 9));
}
```

새로운 테스트코드 추가
소인수분해 9일때 정답은 [3, 3]이어야함



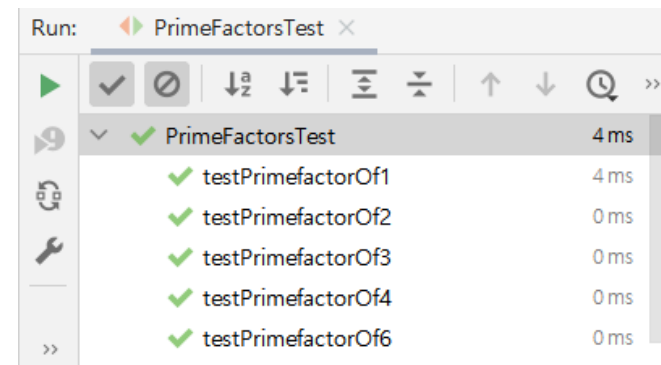
Fail을 눈으로 확인

Green – 하드코딩으로 Green 만들기

✓Green – 테스트케이스 성공

Production

```
else if (number == 6) {  
    for (divisor = 2; number > 1; divisor++) {  
        while (number % divisor == 0) {  
            factors.add(divisor);  
            number /= divisor;  
        }  
    }  
}  
  
else if (number == 9) {  
    factors.add(3);  
    factors.add(3);  
}  
  
else {  
    factors.add(number);  
}
```



Green 눈으로 확인하기

number == 9 일때 코드 추가

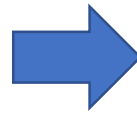
리팩토링 - 중복 코드 제거하기

✓Refactoring - 중복제거

Production

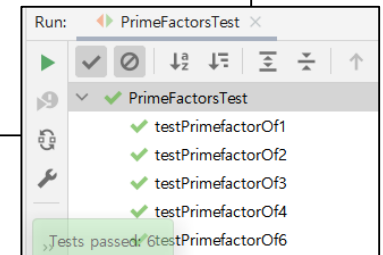
```
else if (number == 6) {  
    for (divisor = 2; number > 1; divisor++) {  
        while (number % divisor == 0) {  
            factors.add(divisor);  
            number /= divisor;  
        }  
    }  
}  
  
else if (number == 9) {  
    factors.add(3);  
    factors.add(3);  
}  
  
else {  
    factors.add(number);  
}  
}
```

기존코드 (사각 영역, 소스코드 제거)



```
else if (number == 6 || number == 9) {  
    for (divisor = 2; number > 1; divisor++) {  
        while (number % divisor == 0) {  
            factors.add(divisor);  
            number /= divisor;  
        }  
    }  
}  
  
else {  
    factors.add(number);  
}
```

코드 추가하기



리팩토링 - 중복 코드 제거하기

✓Refactoring - 중복제거

Production

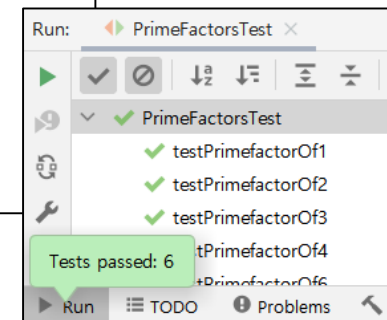
```
public class PrimeFactor {
    public List<Integer> of(int number) {
        List<Integer> factors = new ArrayList<>();
        if (number > 1) {
            int divisor = 2;
            if (number == 4) {
                while (number % divisor == 0) {
                    factors.add(divisor);
                    number /= divisor;
                }
            }
            else if (number == 6 || number == 9) {
                for (divisor = 2; number > 1; divisor++) {
                    while (number % divisor == 0) {
                        factors.add(divisor);
                        number /= divisor;
                    }
                }
            }
            else {
                factors.add(number);
            }
        }
        return factors;
    }
}
```

기존코드 (사각 영역, 소스코드 제거)



```
public class PrimeFactor {
    public List<Integer> of(int number) {
        List<Integer> factors = new ArrayList<>();
        if (number > 1) {
            int divisor = 2;
            if (number == 4 || number == 6 || number == 9) {
                for (divisor = 2; number > 1; divisor++) {
                    while (number % divisor == 0) {
                        factors.add(divisor);
                        number /= divisor;
                    }
                }
            }
            else {
                factors.add(number);
            }
        }
        return factors;
    }
}
```

else if를 if로 변경하고,
number == 4 || 내용 추가.



Prime Factors 실습(07/07) : TDD Cycle 7

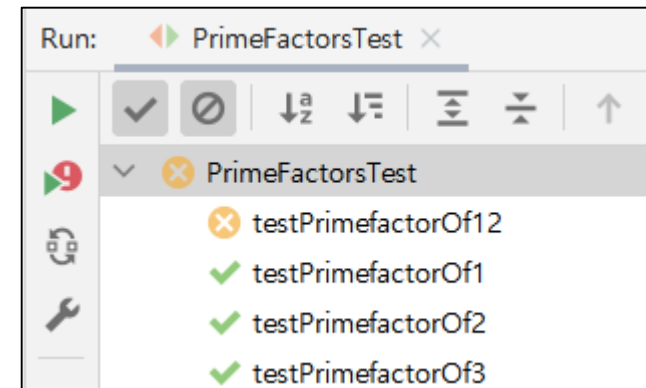
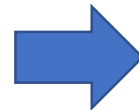
✓Red – '12'를 소인수분해 하는 테스트케이스 작성

Test

```
@Test
public void testPrimefactorOf9() {
    Assert.assertEquals(Arrays.asList(3,3), primefactor.of(number: 9));
}

@Test
public void testPrimefactorOf12() {
    Assert.assertEquals(Arrays.asList(2,2,3), primefactor.of(number: 12));
}
```

12 소인수분해시
[2, 2, 3]이 나와야 한다는 테스트코드 추가.



Fail을 눈으로 확인

Green – 12일때 소스코드 추가.

✓Green – 테스트케이스 성공

Production

```
public class PrimeFactor {  
    public List<Integer> of(int number) {  
        List<Integer> factors = new ArrayList<>();  
        if (number > 1) {  
            int divisor = 2;  
            if (number == 4 || number == 6 || number == 9 || number == 12) {  
                for (divisor = 2; number > 1; divisor++) {  
                    while (number % divisor == 0) {  
                        factors.add(divisor);  
                        number /= divisor;  
                    }  
                }  
            }  
            else {  
                factors.add(number);  
            }  
        }  
        return factors;  
    }  
}
```

Run: PrimeFactorsTest

- ✓ testPrimefactorOf12
- ✓ testPrimefactorOf1
- ✓ testPrimefactorOf2
- ✓ testPrimefactorOf3

number == 12 일때 코드 추가 후 Green 확인하기

리팩토링

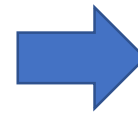
✓Refactoring – 중복코드 제거, generalize

```
import java.util.ArrayList;
import java.util.List;
```

```
public class PrimeFactor {
    public List<Integer> of(int number) {
        List<Integer> factors = new ArrayList<>();

        if (number > 1) {
            int divisor = 2;
            if (number == 4 || number == 6 || number == 9 || number == 12) {
                for (divisor = 2; number > 1; divisor++) {
                    while (number % divisor == 0) {
                        factors.add(divisor);
                        number /= divisor;
                    }
                }
            }
            else {
                factors.add(number);
            }
        }
        return factors;
    }
}
```

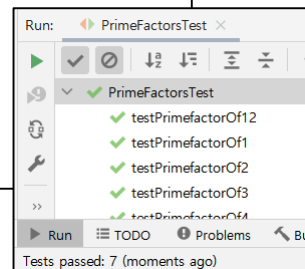
Production



```
import java.util.ArrayList;
import java.util.List;

public class PrimeFactor {
    public List<Integer> of(int number) {
        List<Integer> factors = new ArrayList<>();
        for (int divisor = 2; number > 1; divisor++) {
            while (number % divisor == 0) {
                factors.add(divisor);
                number /= divisor;
            }
        }
        return factors;
    }
}
```

for (int divisor = 2 .. 로 수정)
그리고 최종 테스트



기존코드 (사각 영역, 소스코드 제거)

감사합니다.